MediCare

Healthcare Management System

DBMS Project Report

Database: MySQL Server 8.0.16+ | Tool: MySQL Workbench 8.0+

Submitted by	
Name	Aryan Vaid
UID	25BCS10791
Partner Name	Ansh
Partner UID	25BCS11004
Section	802(A)
Course	Database Management Systems (Academic Project)
Session	2026

Table of Contents

1. Abstract 3
2. Introduction 4
3. Problem Statement and Objectives 5
4. Scope of the Project 6
5. Requirement Analysis 7
6. ER Diagram 8
7. Relational Schema / Database Design 10
8. Normalization (1NF, 2NF, 3NF) 11
9. SQL Implementation (DDL, DML, DQL, TCL, DCL) 12
10. SQL Queries with Explanations and Outputs 14
11. Constraints, Joins, Subqueries and Aggregate Functions 27
12. Screenshots of Execution 28
13. Conclusion 31
14. Future Scope 32
15. References / Research Sources 33

1. Abstract

MediCare is a database-driven Healthcare Management System designed for a small hospital. It models the patient journey from registration and appointment booking through diagnosis, prescription, billing and payment. The project uses MySQL Server 8.0.16 or later and contains ten relational tables protected by primary keys, foreign keys, unique constraints and CHECK constraints. It includes 41 query/output demonstrations and executable examples of DDL, DML, DQL, TCL and DCL.

The system also provides doctor availability checking, complete patient medical history through multi-table joins, and hospital analytics such as top doctors, department-wise patient counts, monthly revenue and most prescribed medicines. All data in the project is fictional.

2. Introduction

Hospitals handle large volumes of patient, doctor, appointment and billing information. When this information is stored on paper or in scattered files it is slow to search, easy to lose and hard to analyse. A relational database stores the data once, links it through keys and answers questions quickly with SQL.

MediCare models a small hospital workflow: a new patient is registered, books an appointment with a doctor, the doctor records a diagnosis and issues a prescription, the hospital generates a bill, and the administrator retrieves histories, schedules and revenue figures.

Technology used

Component	Technology
Database	MySQL Server 8.0.16 or later
Interface	MySQL Workbench
SQL implementation	MySQL SQL scripts (10 files)
ER diagrams	Graphviz / diagrams.net
Documentation	MS Word
Repository	GitHub
Optional application	Java + JDBC

3. Problem Statement and Objectives

Problem statement

Manual record keeping in a hospital leads to duplicated data, double-booked doctors, lost prescriptions and billing errors. There is a need for a structured database that stores hospital data consistently, prevents invalid entries and allows quick retrieval of information.

Objectives

- Design a normalized relational database for hospital operations.
- Store patient, doctor, appointment, medical, prescription and billing data.
- Prevent double booking of a doctor using constraints.
- Retrieve a patient's complete medical history with JOIN queries.
- Generate analytics: busiest doctors, department-wise patients, monthly revenue, most prescribed medicines.
- Demonstrate DDL, DML, DQL, TCL and DCL commands with outputs.

4. Scope of the Project

The project covers six modules:

Module	Description
Patient Management	Register patients, update information, search records, retrieve histories
Doctor Management	Doctor details, specializations, contacts, department assignments
Appointment Management	Schedule appointments, check availability, manage status
Medical Records	Diagnoses, consultation notes, treatment history
Prescription Management	Prescriptions, medicine details, dosage and duration
Billing Management	Bills, payments, payment status

Out of scope

- Insurance claim processing, pharmacy stock, inpatient ward management and payroll.
- A graphical user interface (optional future work with Java + JDBC). This submission is a database prototype, not a production hospital application; it is intended for fictional data only.

5. Requirement Analysis

Functional requirements

- Register, update and search patients.
- Add doctors and assign them to departments.
- Book an appointment only if the doctor's slot is free; cancelling an appointment releases that slot for reuse.
- Record one diagnosis per appointment.
- Issue prescriptions with several medicines, dosage and duration.
- Generate bills for completed appointments, record one or more payments, and reconcile payment status against the total received.
- Produce administrator reports.

Non-functional requirements

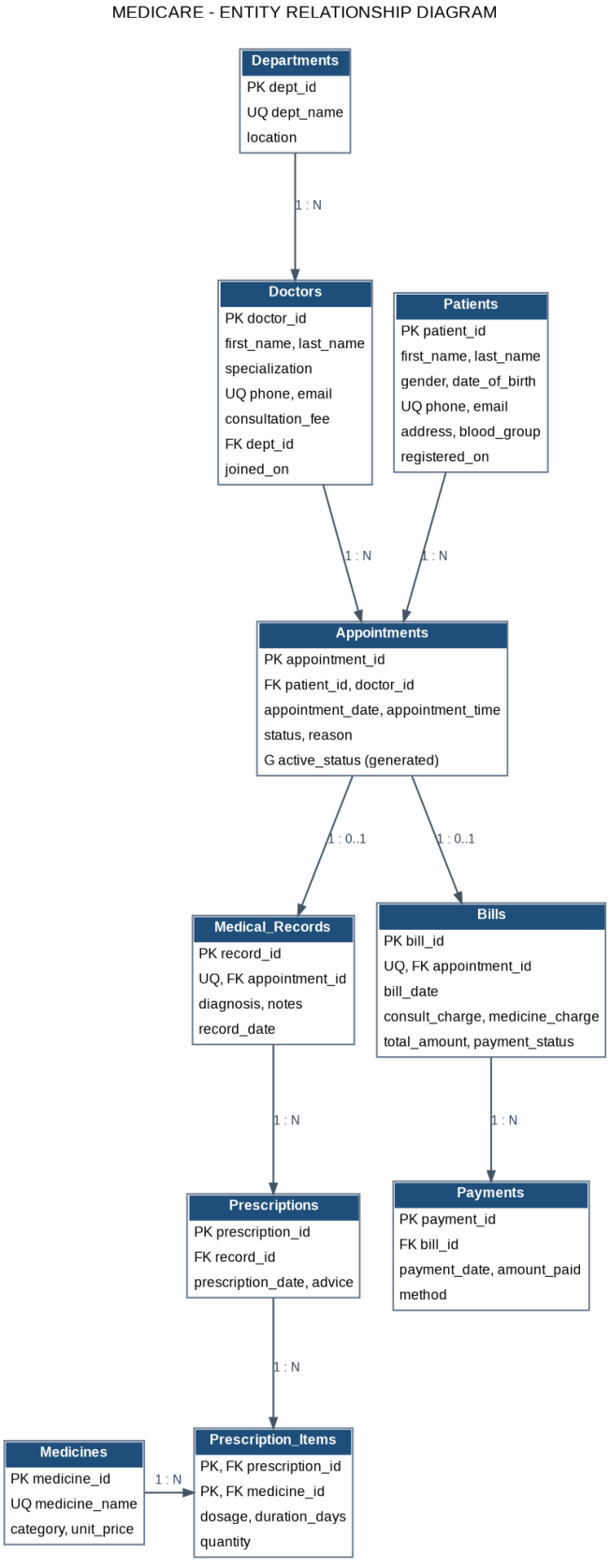
- Data integrity through PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT.
- Normalization up to 3NF to avoid redundancy.
- Reproducible setup: scripts run in numbered order.
- Role-based access using GRANT and REVOKE.

Software and hardware requirements

Item	Requirement
Operating system	Windows 10/11, Linux or macOS
Database server	MySQL Server 8.0.16 or later
Client	MySQL Workbench 8.0 or later
RAM / disk	4 GB RAM, 1 GB free disk

6. ER Diagram

The ER diagram shows ten entities, key attributes (PK/FK/UQ) and relationship cardinalities. G marks the generated active_status column, which is NULL for cancelled appointments and allows a cancelled slot to be reused.



Relationships

Entity A	Relationship	Entity B	Cardinality
Departments	employs	Doctors	1 : N
Patients	books	Appointments	1 : N
Doctors	handles	Appointments	1 : N
Appointments	generates	Medical_Records	1 : 0..1
Medical_Records	has	Prescriptions	1 : N
Prescriptions	contains	Medicines (via Prescription_Items)	M : N
Appointments	generates	Bills	1 : 0..1
Bills	settled by	Payments	1 : N

7. Relational Schema / Database Design

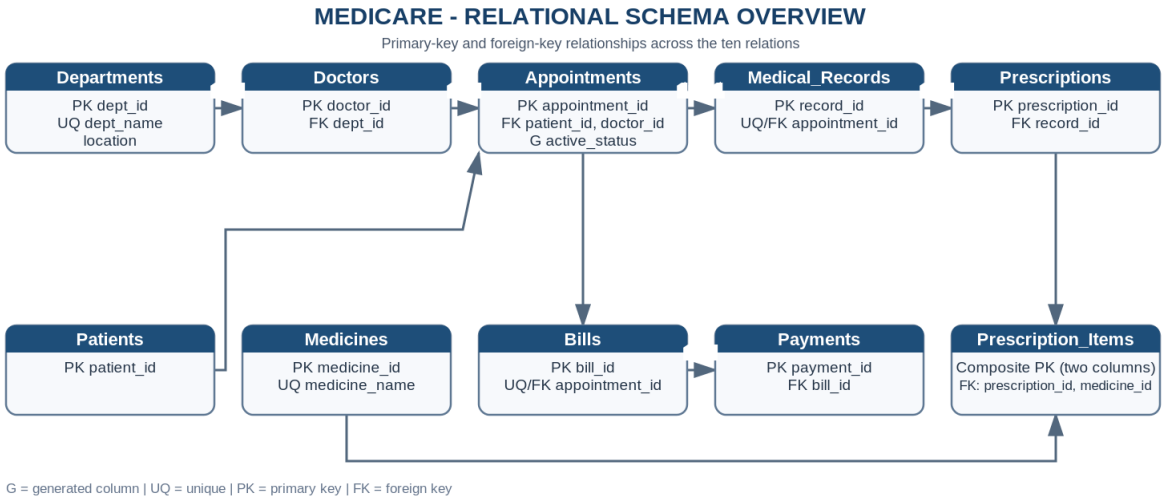


Table	Primary key	Foreign keys	Other attributes
Departments	dept_id	–	dept_name, location
Doctors	doctor_id	dept_id → Departments	first_name, last_name, specialization, phone, email, consultation_fee, joined_on
Patients	patient_id	–	first_name, last_name, gender, date_of_birth, phone, email, address, blood_group, registered_on
Appointments	appointment_id	patient_id → Patients; doctor_id → Doctors	appointment_date, appointment_time, status, reason, active_status (generated)
Medical_Records	record_id	appointment_id → Appointments	diagnosis, notes, record_date
Medicines	medicine_id	–	medicine_name, category, unit_price
Prescriptions	prescription_id	record_id → Medical_Records	prescription_date, advice
Prescription_Items	(prescription_id, medicine_id)	prescription_id → Prescriptions; medicine_id → Medicines	dosage, duration_days, quantity
Bills	bill_id	appointment_id → Appointments	bill_date, consult_charge, medicine_charge, total_amount, payment_status
Payments	payment_id	bill_id → Bills	payment_date, amount_paid, method

8. Normalization (1NF, 2NF, 3NF)

First Normal Form (1NF)

All attributes hold single atomic values and each table has a primary key. A prescription does not store a list of medicines in one column; each medicine is a separate row in Prescription_Items.

Second Normal Form (2NF)

No non-key attribute depends on only part of a composite key. In Prescription_Items (key = prescription_id + medicine_id), dosage, duration_days and quantity depend on the full key. Medicine name and price are kept in Medicines, not repeated here.

Third Normal Form (3NF)

No non-key attribute depends on another non-key attribute. Doctor details are stored only in Doctors, department details only in Departments, and patient details only in Patients; Appointments stores just their IDs. Examples of functional dependencies are patient_id -> first_name, last_name, phone; doctor_id -> first_name, specialization, dept_id; and (prescription_id, medicine_id) -> dosage, duration_days, quantity.

Design note: stored bill amounts

Bills stores consult_charge, medicine_charge and total_amount. These are derived values, kept deliberately so that a bill remains correct even if a doctor's fee or a medicine price changes later (a billing snapshot). This is a controlled exception to strict 3NF.

9. SQL Implementation (DDL, DML, DQL, TCL, DCL)

All scripts are in the SQL folder and are run in numbered order (see README.md). Scripts 01-04 build and seed the database; scripts 05-10 demonstrate queries, transactions, stored-procedure booking and permissions.

Category	File	Commands demonstrated
DDL	01, 02, 03	CREATE DATABASE/TABLE/INDEX/VIEW, generated column, ALTER, DROP
DML	04, 05	INSERT, UPDATE, DELETE
DQL	05 – 08	SELECT with WHERE, ORDER BY, DISTINCT, joins, subqueries, aggregates
TCL + procedure	09	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT; CREATE PROCEDURE, CALL
DCL	10	CREATE USER, GRANT, REVOKE

DDL example: Appointments table

```
CREATE TABLE Appointments (
    appointment_id   INT AUTO_INCREMENT PRIMARY KEY,
    patient_id       INT NOT NULL,
    doctor_id        INT NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status           ENUM('Scheduled','Completed','Cancelled') NOT NULL DEFAULT 'Scheduled',
    reason           VARCHAR(150),
    active_status    VARCHAR(10) GENERATED ALWAYS AS (
                     CASE WHEN status = 'Cancelled' THEN NULL ELSE 'Active' END) STORED,
    CONSTRAINT fk_app_patient FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_app_doctor FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_doctor_slot UNIQUE (
        doctor_id, appointment_date, appointment_time, active_status)
);
```

DDL example: ALTER and DROP

```
-- ALTER: add a column
ALTER TABLE Patients ADD COLUMN emergency_contact VARCHAR(15);

-- ALTER: modify a column
ALTER TABLE Patients MODIFY COLUMN address VARCHAR(200);

-- ALTER: add a CHECK constraint
ALTER TABLE Patients
    ADD CONSTRAINT chk_dob CHECK (date_of_birth >= '1900-01-01');
```

DML example

```
-- INSERT a new patient (patient registration step of the workflow)
INSERT INTO Patients
(first_name,last_name,gender,date_of_birth,phone,email,address,blood_group)
VALUES ('Test','Patient','Other','2000-01-01','9900009999','test.patient@mail.test','Demo
Address','O+');

-- UPDATE: change the new patient's phone number and address
UPDATE Patients
SET phone = '9900008888', address = 'Updated Demo Address'
WHERE phone = '9900009999';
```

TCL example

```
-- T3. SAVEPOINT: undo only part of a transaction
START TRANSACTION;

UPDATE Doctors SET consultation_fee = 850 WHERE doctor_id = 7;
SAVEPOINT after_fee_update;

UPDATE Doctors SET consultation_fee = 1 WHERE doctor_id = 8;    -- wrong update
ROLLBACK TO SAVEPOINT after_fee_update;                          -- undo only the wrong update

COMMIT;
```

DCL example

```
-- 1. Create users (demo passwords only)
CREATE USER 'receptionist'@'localhost' IDENTIFIED BY 'Recep@123';
CREATE USER 'doctor_user'@'localhost'  IDENTIFIED BY 'Doctor@123';
CREATE USER 'admin_user'@'localhost'   IDENTIFIED BY 'Admin@123';

-- 2. GRANT privileges
-- Receptionist: register patients and book appointments only
GRANT SELECT, INSERT, UPDATE ON medicare_db.Patients     TO 'receptionist'@'localhost';
GRANT SELECT, INSERT, UPDATE ON medicare_db.Appointments TO 'receptionist'@'localhost';
GRANT SELECT ON medicare_db.Doctors TO 'receptionist'@'localhost';

-- Doctor: read patients, write medical records and prescriptions
GRANT SELECT ON medicare_db.Patients  TO 'doctor_user'@'localhost';
GRANT SELECT ON medicare_db.Appointments TO 'doctor_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON medicare_db.Medical_Records     TO 'doctor_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON medicare_db.Prescriptions       TO 'doctor_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON medicare_db.Prescription_Items  TO 'doctor_user'@'localhost';
GRANT SELECT ON medicare_db.Medicines TO 'doctor_user'@'localhost';

-- Administrator: full control of this database
GRANT ALL PRIVILEGES ON medicare_db.* TO 'admin_user'@'localhost';

FLUSH PRIVILEGES;
```

Implementation note: The stored procedure checks availability, relies on the UNIQUE key as the final concurrency safeguard, and demonstrates rebooking after cancellation. Script 09 also reconciles payment_status from recorded Payments; this is an explicit update, not an automatic trigger on every payment INSERT. DCL output is shown with SHOW GRANTS (see Section 12).

10. SQL Queries with Explanations and Outputs

Each query is shown with its SQL, a short explanation and the output obtained on the sample data.

Q1. List all patients

Lists every registered patient with basic details.

```
SELECT patient_id, first_name, last_name, gender, phone FROM Patients
```

patient_id	first_name	last_name	gender	phone
1	Aarav	Singh	Male	9900000001
2	Diya	Patel	Female	9900000002
3	Kabir	Khanna	Male	9900000003
4	Ishita	Gupta	Female	9900000004
5	Rahul	Verma	Male	9900000005
6	Meera	Joshi	Female	9900000006
7	Aditya	Reddy	Male	9900000007
8	Sana	Ali	Female	9900000008
9	Vivek	Bansal	Male	9900000009
10	Tanya	Chopra	Female	9900000010
11	Harsh	Vardhan	Male	9900000011
12	Pooja	Malik	Female	9900000012
13	Nikhil	Arora	Male	9900000013
14	Riya	Saxena	Female	9900000014
15	Manoj	Tiwari	Male	9900000015

Q2. Patients with blood group O+

Filters patients having blood group O+ using WHERE.

```
SELECT patient_id, first_name, last_name, blood_group
FROM Patients
WHERE blood_group = 'O+'
```

patient_id	first_name	last_name	blood_group
2	Diya	Patel	O+
8	Sana	Ali	O+
12	Pooja	Malik	O+
15	Manoj	Tiwari	O+

Q3. Doctors whose consultation fee is above 700

Shows doctors whose consultation fee is greater than 700.

```
SELECT doctor_id, first_name, last_name, specialization, consultation_fee
FROM Doctors
WHERE consultation_fee > 700
```

doctor_id	first_name	last_name	specialization	consultation_fee
1	Rohan	Mehta	Cardiologist	800.00
2	Anita	Sharma	Cardiologist	900.00
3	Vikram	Rao	Neurologist	1000.00

doctor_id	first_name	last_name	specialization	consultation_fee
4	Sneha	Kapoor	Orthopedic Surgeon	850.00
5	Arjun	Nair	Orthopedic Surgeon	750.00

Q4. Appointments that are still scheduled

Finds appointments that are still scheduled (not yet completed or cancelled).

```
SELECT appointment_id, patient_id, doctor_id, appointment_date, appointment_time
FROM Appointments
WHERE status = 'Scheduled'
```

appointment_id	patient_id	doctor_id	appointment_date	appointment_time
23	5	1	2026-10-20	09:30:00
25	14	2	2026-11-18	10:00:00

Q5. Patients registered between 1 Aug 2026 and 31 Aug 2026 (BETWEEN)

Uses BETWEEN to list patients registered in August 2026.

```
SELECT patient_id, first_name, last_name, registered_on
FROM Patients
WHERE registered_on BETWEEN '2026-08-01' AND '2026-08-31'
```

patient_id	first_name	last_name	registered_on
7	Aditya	Reddy	2026-08-01
8	Sana	Ali	2026-08-03
9	Vivek	Bansal	2026-08-09
10	Tanya	Chopra	2026-08-12
11	Harsh	Vardhan	2026-08-20

Q6. Search patients whose first name starts with 'A' (LIKE)

Uses LIKE with a wildcard to find patients whose first name starts with 'A'.

```
SELECT patient_id, first_name, last_name
FROM Patients
WHERE first_name LIKE 'A%'
```

patient_id	first_name	last_name
1	Aarav	Singh
7	Aditya	Reddy

Q7. Doctors sorted by fee, highest first (ORDER BY)

Sorts doctors by fee (highest first); ties are sorted by last name.

```
SELECT first_name, last_name, specialization, consultation_fee
FROM Doctors
ORDER BY consultation_fee DESC, last_name ASC
```

first_name	last_name	specialization	consultation_fee
Vikram	Rao	Neurologist	1000.00
Anita	Sharma	Cardiologist	900.00
Sneha	Kapoor	Orthopedic Surgeon	850.00

first_name	last_name	specialization	consultation_fee
Rohan	Mehta	Cardiologist	800.00
Arjun	Nair	Orthopedic Surgeon	750.00
Priya	Iyer	Pediatrician	600.00
Karan	Malhotra	General Physician	500.00
Neha	Verma	General Physician	500.00

Q8. Distinct doctor specializations (DISTINCT)

DISTINCT removes duplicate specializations.

```
SELECT DISTINCT specialization
FROM Doctors
ORDER BY specialization
```

specialization
Cardiologist
General Physician
Neurologist
Orthopedic Surgeon
Pediatrician

Q9. Distinct blood groups present among patients

DISTINCT returns each blood group once.

```
SELECT DISTINCT blood_group
FROM Patients
ORDER BY blood_group
```

blood_group
A+
A-
AB+
AB-
B+
B-
O+
O-

Q10. Appointments with status Completed or Cancelled (IN)

IN selects appointments that are Completed or Cancelled; the latest five are shown.

```
SELECT appointment_id, appointment_date, status
FROM Appointments
WHERE status IN ('Completed','Cancelled')
ORDER BY appointment_date DESC
LIMIT 5
```

appointment_id	appointment_date	status
24	2026-10-22	Cancelled
22	2026-10-15	Completed

appointment_id	appointment_date	status
21	2026-10-10	Completed
20	2026-10-06	Completed
19	2026-10-01	Completed

DML: after UPDATE

Shows the test patient after UPDATE changed phone and address.

```
SELECT patient_id, first_name, phone, address FROM Patients WHERE first_name = 'Test'
```

patient_id	first_name	phone	address
16	Test	9900008888	Updated Demo Address

DML: after DELETE

Patient count after DELETE – back to 15, so sample data is unchanged.

```
SELECT COUNT(*) AS patients_after_delete FROM Patients
```

patients_after_delete
15

J1. Appointment list with patient and doctor names (INNER JOIN, 3 tables)

Joins Appointments, Patients and Doctors to display readable names instead of IDs.

```
SELECT a.appointment_id,
       CONCAT(p.first_name,' ',p.last_name) AS patient,
       CONCAT('Dr. ',d.first_name,' ',d.last_name) AS doctor,
       a.appointment_date, a.status
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
JOIN Doctors  d ON a.doctor_id  = d.doctor_id
ORDER BY a.appointment_date
LIMIT 8
```

appointment_id	patient	doctor	appointment_date	status
1	Aarav Singh	Dr. Rohan Mehta	2026-08-04	Completed
2	Diya Patel	Dr. Karan Malhotra	2026-08-05	Completed
3	Kabir Khanna	Dr. Sneha Kapoor	2026-08-06	Completed
4	Ishita Gupta	Dr. Priya Iyer	2026-08-08	Completed
5	Rahul Verma	Dr. Anita Sharma	2026-08-12	Completed
6	Meera Joshi	Dr. Vikram Rao	2026-08-14	Completed
7	Aditya Reddy	Dr. Priya Iyer	2026-08-18	Completed
8	Sana Ali	Dr. Neha Verma	2026-08-20	Completed

J2. Doctors with their department (INNER JOIN)

Joins Doctors with Departments to show where each doctor works.

```
SELECT CONCAT('Dr. ',d.first_name,' ',d.last_name) AS doctor,
       d.specialization, dep.dept_name, dep.location
FROM Doctors d
JOIN Departments dep ON d.dept_id = dep.dept_id
ORDER BY dep.dept_name
```

doctor	specialization	dept_name	location
Dr. Rohan Mehta	Cardiologist	Cardiology	Block A
Dr. Anita Sharma	Cardiologist	Cardiology	Block A
Dr. Karan Malhotra	General Physician	General Medicine	Main Block
Dr. Neha Verma	General Physician	General Medicine	Main Block
Dr. Vikram Rao	Neurologist	Neurology	Block B
Dr. Sneha Kapoor	Orthopedic Surgeon	Orthopedics	Block C
Dr. Arjun Nair	Orthopedic Surgeon	Orthopedics	Block C
Dr. Priya Iyer	Pediatrician	Pediatrics	Block D

J3. All appointments and their bills; appointments without a bill show NULL (LEFT JOIN)

LEFT JOIN keeps all appointments; those with no bill (scheduled/cancelled) show NULL.

```
SELECT a.appointment_id, a.appointment_date, a.status, b.bill_id, b.total_amount
FROM Appointments a
LEFT JOIN Bills b ON a.appointment_id = b.appointment_id
WHERE b.bill_id IS NULL
```

appointment_id	appointment_date	status	bill_id	total_amount
23	2026-10-20	Scheduled	NULL	NULL
24	2026-10-22	Cancelled	NULL	NULL
25	2026-11-18	Scheduled	NULL	NULL

J4. COMPLETE MEDICAL HISTORY of patient 1 (feature: Patient medical history)

Patient medical history: a seven-table JOIN from patient to appointment, diagnosis, prescription and medicines.

```
SELECT a.appointment_date AS visit_date,
       CONCAT('Dr. ',d.first_name,' ',d.last_name) AS doctor,
       mr.diagnosis,
       m.medicine_name, pi.dosage, pi.duration_days
FROM Patients p
JOIN Appointments a       ON a.patient_id = p.patient_id
JOIN Doctors d            ON a.doctor_id = d.doctor_id
LEFT JOIN Medical_Records mr   ON mr.appointment_id = a.appointment_id
LEFT JOIN Prescriptions pr     ON pr.record_id = mr.record_id
LEFT JOIN Prescription_Items pi ON pi.prescription_id = pr.prescription_id
LEFT JOIN Medicines m          ON m.medicine_id = pi.medicine_id
WHERE p.patient_id = 1
ORDER BY a.appointment_date
```

visit_date	doctor	diagnosis	medicine_name	dosage	duration_days
2026-08-04	Dr. Rohan Mehta	Hypertension	Amlodipine 5mg	1-0-0	30
2026-08-04	Dr. Rohan Mehta	Hypertension	Aspirin 75mg	0-1-0	30
2026-09-02	Dr. Anita Sharma	Hypertension (controlled)	Amlodipine 5mg	1-0-0	30

J5. Outstanding balance per bill (LEFT JOIN with Payments + GROUP BY)

Calculates the unpaid balance of every bill using LEFT JOIN, SUM, GROUP BY and HAVING.

```
SELECT b.bill_id, b.total_amount,
       COALESCE(SUM(py.amount_paid),0) AS paid,
       b.total_amount - COALESCE(SUM(py.amount_paid),0) AS balance
FROM Bills b
LEFT JOIN Payments py ON py.bill_id = b.bill_id
GROUP BY b.bill_id, b.total_amount
HAVING balance > 0
```

bill_id	total_amount	paid	balance
5	1335.00	667.50	667.50
9	1190.00	0.00	1190.00
13	1970.00	985.00	985.00
22	1110.00	0.00	1110.00

J6. Pairs of doctors working in the same department (SELF JOIN)

SELF JOIN pairs doctors who work in the same department.

```
SELECT d1.last_name AS doctor_1, d2.last_name AS doctor_2, dep.dept_name
FROM Doctors d1
JOIN Doctors d2 ON d1.dept_id = d2.dept_id AND d1.doctor_id < d2.doctor_id
JOIN Departments dep ON dep.dept_id = d1.dept_id
```

doctor_1	doctor_2	dept_name
Mehta	Sharma	Cardiology
Kapoor	Nair	Orthopedics
Malhotra	Verma	General Medicine

J7. Prescription of appointment 1: medicines, dosage and cost

Shows the medicines, dosage and line cost of the prescription issued for appointment 1.

```
SELECT m.medicine_name, pi.dosage, pi.quantity, m.unit_price,
       pi.quantity * m.unit_price AS line_total
FROM Medical_Records mr
JOIN Prescriptions pr      ON pr.record_id = mr.record_id
JOIN Prescription_Items pi ON pi.prescription_id = pr.prescription_id
JOIN Medicines m           ON m.medicine_id = pi.medicine_id
WHERE mr.appointment_id = 1
```

medicine_name	dosage	quantity	unit_price	line_total
Amlodipine 5mg	1-0-0	30	4.00	120.00
Aspirin 75mg	0-1-0	30	1.50	45.00

S1. Doctors charging more than the average consultation fee

Subquery returns the average fee; the outer query lists doctors charging more.

```
SELECT first_name, last_name, consultation_fee
FROM Doctors
WHERE consultation_fee > (SELECT AVG(consultation_fee) FROM Doctors)
```

first_name	last_name	consultation_fee
Rohan	Mehta	800.00
Anita	Sharma	900.00

first_name	last_name	consultation_fee
Vikram	Rao	1000.00
Sneha	Kapoor	850.00
Arjun	Nair	750.00

S2. Patients who have consulted a Cardiology doctor (IN subquery)

IN subquery finds patients who have visited a Cardiology doctor.

```
SELECT patient_id, first_name, last_name
FROM Patients
WHERE patient_id IN (
    SELECT a.patient_id
    FROM Appointments a
    JOIN Doctors d ON a.doctor_id = d.doctor_id
    JOIN Departments dep ON d.dept_id = dep.dept_id
    WHERE dep.dept_name = 'Cardiology')
```

patient_id	first_name	last_name
1	Aarav	Singh
5	Rahul	Verma
9	Vivek	Bansal
14	Riya	Saxena
15	Manoj	Tiwari

S3. Patients with at least one unpaid bill (EXISTS, correlated)

Correlated EXISTS finds patients having at least one unpaid bill.

```
SELECT p.patient_id, p.first_name, p.last_name
FROM Patients p
WHERE EXISTS (
    SELECT 1
    FROM Appointments a
    JOIN Bills b ON b.appointment_id = a.appointment_id
    WHERE a.patient_id = p.patient_id AND b.payment_status = 'Unpaid')
```

patient_id	first_name	last_name
9	Vivek	Bansal
10	Tanya	Chopra

S4. Most expensive medicine (subquery with MAX)

Subquery with MAX finds the most expensive medicine.

```
SELECT medicine_name, unit_price
FROM Medicines
WHERE unit_price = (SELECT MAX(unit_price) FROM Medicines)
```

medicine_name	unit_price
Vitamin D3 60K	25.00

S5. Bills above the average bill amount

Lists bills above the average bill amount.

```
SELECT bill_id, appointment_id, total_amount
FROM Bills
WHERE total_amount > (SELECT AVG(total_amount) FROM Bills)
ORDER BY total_amount DESC
```

bill_id	appointment_id	total_amount
13	13	1970.00
3	3	1670.00
5	5	1335.00
6	6	1190.00
9	9	1190.00
16	16	1115.00
22	22	1110.00
19	19	1065.00
11	11	1020.00
1	1	965.00

S6. DOCTOR AVAILABILITY CHECK (feature)

Doctor availability check: returns 'Not available' because Dr. Rohan Mehta already has an appointment at that slot.

```
SELECT CASE
         WHEN EXISTS (SELECT 1 FROM Appointments
                      WHERE doctor_id = 1
                        AND appointment_date = '2026-08-04'
                        AND appointment_time = '10:00:00'
                        AND status <> 'Cancelled')
         THEN 'Not available'
         ELSE 'Available'
       END AS slot_status
```

slot_status
Not available

S7. Doctors who have never had a cancelled appointment (NOT IN)

NOT IN lists doctors with no cancelled appointment.

```
SELECT doctor_id, first_name, last_name
FROM Doctors
WHERE doctor_id NOT IN (SELECT doctor_id FROM Appointments WHERE status = 'Cancelled')
```

doctor_id	first_name	last_name
1	Rohan	Mehta
2	Anita	Sharma
3	Vikram	Rao
4	Sneha	Kapoor
5	Arjun	Nair
6	Priya	Iyer
8	Neha	Verma

S8. Patients with more appointments than the average per patient (subquery in HAVING)

Subquery inside HAVING lists patients who visited more often than the per-patient average.

```
SELECT patient_id, COUNT(*) AS visits
FROM Appointments
GROUP BY patient_id
HAVING COUNT(*) > (SELECT COUNT(*) / COUNT(DISTINCT patient_id) FROM Appointments)
```

patient_id	visits
1	2
2	2
3	2
4	2
5	2
6	2
8	2
9	2
10	2
14	2

A1. Total number of patients

COUNT of all patients.

```
SELECT COUNT(*) AS total_patients FROM Patients
```

total_patients
15

A2. Average, minimum and maximum consultation fee

AVG, MIN and MAX of consultation fees.

```
SELECT AVG(consultation_fee) AS avg_fee,
       MIN(consultation_fee) AS min_fee,
       MAX(consultation_fee) AS max_fee
FROM Doctors
```

avg_fee	min_fee	max_fee
737.50	500.00	1000.00

A3. Total billed amount and total amount collected

SUM of billed amount versus amount actually collected.

```
SELECT (SELECT SUM(total_amount) FROM Bills)    AS total_billed,
       (SELECT SUM(amount_paid)  FROM Payments) AS total_collected
```

total_billed	total_collected
20731.00	16778.50

A4. Number of appointments by status (GROUP BY)

GROUP BY status counts appointments in each state.

```
SELECT status, COUNT(*) AS total
FROM Appointments
GROUP BY status
```

status	total
Cancelled	1
Completed	22
Scheduled	2

A5. ANALYTICS: Most frequently consulted doctors

Analytics: doctors ranked by completed consultations.

```
SELECT CONCAT('Dr. ',d.first_name,' ',d.last_name) AS doctor,
       COUNT(*) AS consultations
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.status = 'Completed'
GROUP BY d.doctor_id, d.first_name, d.last_name
ORDER BY consultations DESC, doctor
LIMIT 5
```

doctor	consultations
Dr. Priya Iyer	4
Dr. Anita Sharma	3
Dr. Karan Malhotra	3
Dr. Neha Verma	3
Dr. Rohan Mehta	3

A6. ANALYTICS: Department-wise patient count (distinct patients)

Analytics: number of distinct patients per department.

```
SELECT dep.dept_name, COUNT(DISTINCT a.patient_id) AS patients
FROM Departments dep
JOIN Doctors d       ON d.dept_id = dep.dept_id
JOIN Appointments a  ON a.doctor_id = d.doctor_id
GROUP BY dep.dept_id, dep.dept_name
ORDER BY patients DESC
```

dept_name	patients
Cardiology	5
General Medicine	5
Neurology	3
Pediatrics	3
Orthopedics	2

A7. ANALYTICS: Monthly revenue (payments received)

Analytics: revenue received per month.

```
SELECT DATE_FORMAT(payment_date,'%Y-%m') AS month,
       SUM(amount_paid) AS revenue
FROM Payments
GROUP BY DATE_FORMAT(payment_date,'%Y-%m')
ORDER BY month
```

month	revenue
2026-08	7846.50
2026-09	6562.00
2026-10	2370.00

A8. ANALYTICS: Most prescribed medicines

Analytics: medicines prescribed most often.

```
SELECT m.medicine_name, COUNT(*) AS times_prescribed, SUM(pi.quantity) AS total_units
FROM Prescription_Items pi
JOIN Medicines m ON m.medicine_id = pi.medicine_id
GROUP BY m.medicine_id, m.medicine_name
ORDER BY times_prescribed DESC, total_units DESC
LIMIT 5
```

medicine_name	times_prescribed	total_units
Amlodipine 5mg	5	150
Paracetamol 500mg	5	49
Aspirin 75mg	4	120
Cetirizine 10mg	4	27
Atorvastatin 10mg	3	90

A9. Doctors with at least 3 completed consultations (HAVING)

HAVING keeps only doctors with at least 3 completed consultations.

```
SELECT doctor_id, COUNT(*) AS consultations
FROM Appointments
WHERE status = 'Completed'
GROUP BY doctor_id
HAVING COUNT(*) >= 3
```

doctor_id	consultations
1	3
2	3
3	3
6	4
7	3
8	3

A10. Revenue per department (payments -> bills -> appointments -> doctors -> departments)

Revenue collected per department through a five-table join.

```
SELECT dep.dept_name, SUM(py.amount_paid) AS revenue
FROM Payments py
JOIN Bills b         ON b.bill_id = py.bill_id
JOIN Appointments a  ON a.appointment_id = b.appointment_id
JOIN Doctors d       ON d.doctor_id = a.doctor_id
JOIN Departments dep ON dep.dept_id = d.dept_id
GROUP BY dep.dept_name
HAVING SUM(py.amount_paid) > 1000
ORDER BY revenue DESC
```

dept_name	revenue
Cardiology	4832.50
General Medicine	3654.00
Orthopedics	3385.50
Pediatrics	2731.50
Neurology	2175.00

T1. COMMIT

A uniquely identified demo appointment is inserted and COMMIT permanently saves the booking.

```
SELECT appointment_id, status FROM Appointments WHERE patient_id = 7 AND doctor_id = 6 AND
appointment_date = '2026-11-20' AND appointment_time = '10:00:00' AND reason = 'MediCare TCL
commit demo'
```

appointment_id	status
26	Scheduled

T2. ROLLBACK (inside)

Payment count inside the transaction (one extra row).

```
SELECT COUNT(*) AS payments_inside_transaction FROM Payments
```

payments_inside_transaction
21

T2. ROLLBACK (after)

After ROLLBACK the temporary payment is removed (back to 20).

```
SELECT COUNT(*) AS payments_after_rollback FROM Payments
```

payments_after_rollback
20

T3. SAVEPOINT

Doctor 7's fee stays 850 (kept), doctor 8's wrong update to 1 was rolled back to 500.

```
SELECT doctor_id, consultation_fee FROM Doctors WHERE doctor_id IN (7,8)
```

doctor_id	consultation_fee
7	850.00
8	500.00

11. Constraints, Joins, Subqueries and Aggregate Functions

Concept	Where demonstrated
PRIMARY KEY	All tables; composite key in Prescription_Items
FOREIGN KEY	9 relationships, with ON UPDATE CASCADE / ON DELETE RESTRICT or CASCADE
UNIQUE	phone, email, dept_name, medicine_name; appointment_id in Medical_Records and Bills; (doctor_id, date, time, active_status) prevents active double booking and frees cancelled slots
NOT NULL / DEFAULT	Names, dates; status defaults to 'Scheduled', payment_status to 'Unpaid'
CHECK	consultation_fee > 0, unit_price >= 0, total_amount >= 0, amount_paid > 0, duration_days > 0, quantity > 0, date_of_birth >= 1900, valid blood group
Joins	J1-J7 (INNER, LEFT, SELF; seven-table medical-history join)
Subqueries	S1 – S8 (scalar, IN, NOT IN, EXISTS, subquery in HAVING)
Aggregates	A1 – A10 (COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING)
Transactions	Script 09 (COMMIT, ROLLBACK, SAVEPOINT)
Permissions	Script 10 (GRANT, REVOKE)
Stored procedure	Script 09 (CREATE PROCEDURE, CALL, slot availability and cancellation/rebooking)

12. Screenshots of Execution

Screenshot 1: Tables created (SHOW TABLES) – script 02

Tables_in_medicare_db
appointments
bills
departments
doctors
medical_records
medicines
patients
payments
prescription_items
prescriptions

Screenshot 2: Row counts after INSERT – script 04

tbl	total
Departments	5
Doctors	8
Patients	15
Appointments	25
Medical_Records	22
Medicines	12
Prescriptions	22
Prescription_Items	37
Bills	22
Payments	20

Screenshot 3: Joins output (J4 patient medical history) – script 06

visit_date	doctor	diagnosis	medicine_name	dosage	duration_days
2026-08-04	Dr. Rohan Mehta	Hypertension	Aspirin 75mg	0-1-0	30
2026-08-04	Dr. Rohan Mehta	Hypertension	Amlodipine 5mg	1-0-0	30
2026-09-02	Dr. Anita Sharma	Hypertension (controlled)	Amlodipine 5mg	1-0-0	30

Screenshot 4: Aggregate analytics (A5 – A8) – script 08

```
26  SELECT CONCAT('Dr. ',d.first_name,' ',d.last_name) AS doctor,
27         COUNT(*) AS consultations
28  FROM Appointments a
29  JOIN Doctors d ON a.doctor_id = d.doctor_id
30  WHERE a.status = 'Completed'
31  GROUP BY d.doctor_id, d.first_name, d.last_name
32  ORDER BY consultations DESC, doctor
33  LIMIT 5;
34
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

doctor	consultations
Dr. Priya Iyer	4
Dr. Anita Sharma	3
Dr. Karan Malhotra	3
Dr. Neha Verma	3
Dr. Rohan Mehta	3

```
36  SELECT dep.dept_name, COUNT(DISTINCT a.patient_id) AS patients
37  FROM Departments dep
38  JOIN Doctors d       ON d.dept_id = dep.dept_id
39  JOIN Appointments a  ON a.doctor_id = d.doctor_id
40  GROUP BY dep.dept_id, dep.dept_name
41  ORDER BY patients DESC;
42
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	patients
Cardiology	5
General Medicine	5
Neurology	3
Orthopedics	3
Pediatrics	3

```
44  SELECT DATE_FORMAT(payment_date,'%Y-%m') AS month,
45         SUM(amount_paid) AS revenue
46  FROM Payments
47  GROUP BY DATE_FORMAT(payment_date,'%Y-%m')
48  ORDER BY month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

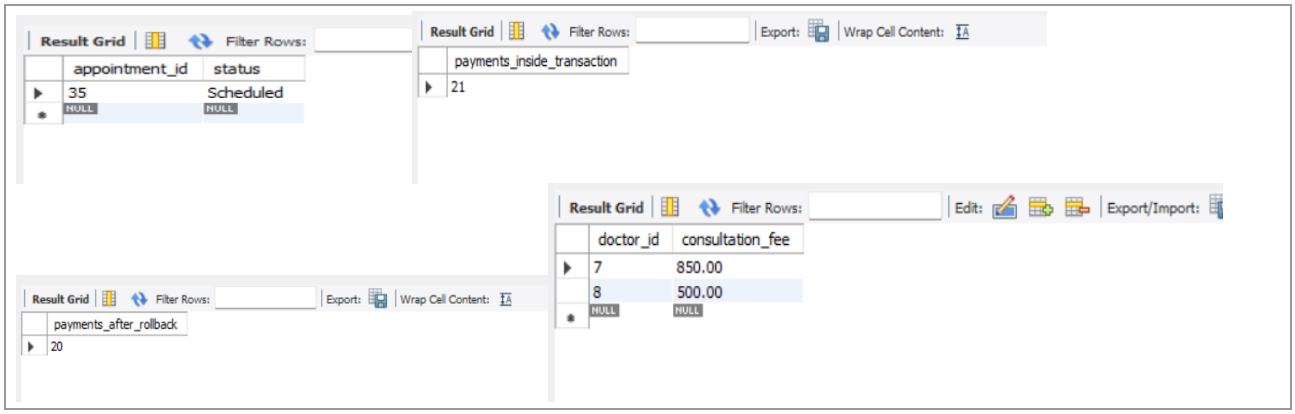
month	revenue
2026-08	7846.50
2026-09	6562.00
2026-10	2370.00

```
51  SELECT m.medicine_name, COUNT(*) AS times_prescribed, SUM(pi.quantity) AS total_units
52  FROM Prescription_Items pi
53  JOIN Medicines m ON m.medicine_id = pi.medicine_id
54  GROUP BY m.medicine_id, m.medicine_name
55  ORDER BY times_prescribed DESC, total_units DESC
56  LIMIT 5;
```

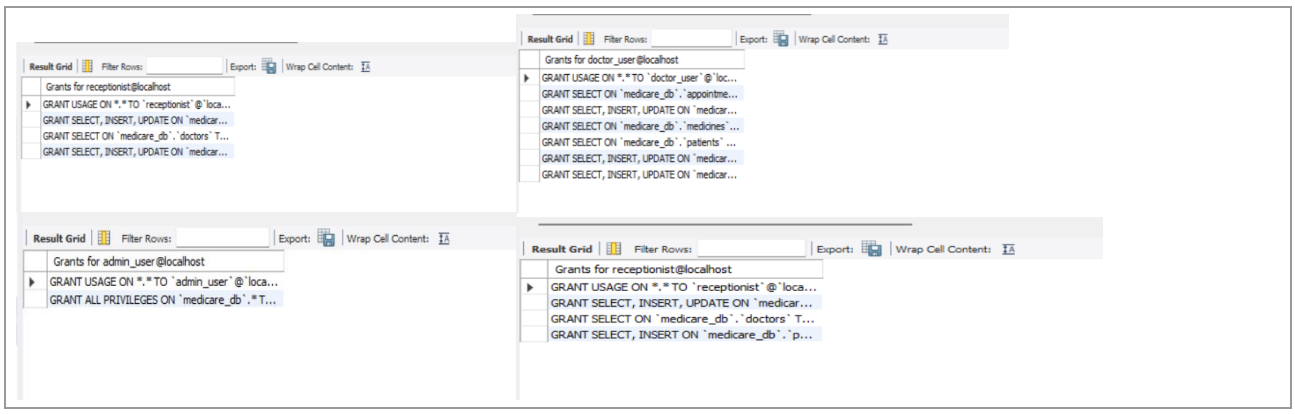
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

medicine_name	times_prescribed	total_units
Amlodipine 5mg	5	150
Paracetamol 500mg	5	49
Aspirin 75mg	4	120
Cetirizine 10mg	4	27
Atorvastatin 10mg	3	90

Screenshot 5: Transaction demo (COMMIT / ROLLBACK / SAVEPOINT and cancellation/rebooking) - script 09



Screenshot 6: SHOW GRANTS output – script 10



13. Conclusion

MediCare demonstrates a normalized hospital database with ten related tables supporting the workflow from patient registration to billing and payment. Primary keys, foreign keys, unique constraints and CHECK constraints protect data integrity. The generated appointment-status key prevents active double bookings while allowing cancelled slots to be reused. JOINs, subqueries, aggregate queries, transactions and access-control examples demonstrate practical SQL use. Payment status is reconciled by an explicit SQL update; automatic trigger-based updates remain future work.

14. Future Scope

- Java + JDBC or web front end for receptionists and doctors.
- Triggers to calculate bill totals and update payment status automatically.
- Pharmacy stock tracking and low-stock alerts.
- Inpatient admission, ward and bed management.
- Insurance claims and online appointment booking.
- Database backup and audit logging.

15. References / Research Sources

- Oracle, MySQL 8.0 Reference Manual (constraints, generated columns, transactions and stored programs) - https://dev.mysql.com/doc/refman/8.0/en/ (accessed 11 November 2026).
- Oracle, MySQL Workbench Manual – https://dev.mysql.com/doc/workbench/en/ (accessed 11 November 2026).
- Elmasri, R. and Navathe, S., Fundamentals of Database Systems, Pearson.
- Silberschatz, A., Korth, H. and Sudarshan, S., Database System Concepts, McGraw-Hill.
- W3Schools SQL Tutorial – https://www.w3schools.com/sql/ (accessed 11 November 2026).
- diagrams.net documentation – https://www.diagrams.net/doc/ (accessed 11 November 2026).